Forum Documentation

Team Members:

Lates Ovidiu-Alexandru
Boca Ioan-Doru
Blaga Alexandra

May 28, 2026

Contents

1 Technology Stack **1**

2 System Architecture **2**
2.1 API Gateway Layer 2
2.2 Microservices Architecture 2
2.2.1 Internal Microservice Layers 2

3 Persistence Layer (MongoDB) **3**
3.1 Core Data Abstractions 3
3.2 Infrastructure & Optimization 3

4 Communication Layer (gRPC) **4**
4.1 Messaging & Contract Layer 4
4.2 Handler Framework 4
4.3 Infrastructure 5

5 Execution Flow: Authentication Example **6**

6 Frontend Architecture & Application Flow **7**
6.1 Core Technologies & Patterns 7
6.2 Authentication & State Management 7
6.3 Core Pages & Components 7
6.3.1 Authentication (Login & Register) 7
6.3.2 Dashboard / Home 7
6.3.3 Question Management (Ask & Detail) 8
6.3.4 User & Role Management (Account & Moderator) 8
6.4 Service Layer & API Integration 8

7 Testing Strategy **9**
7.1 Unit & Integration Testing (Frontend) 9
7.2 End-to-End (E2E) Testing 9
7.3 Automated Quality Assurance 9

1 Technology Stack

The project leverages a modern, distributed ecosystem designed for performance, scalability, and modularity.

- **Frontend**: React (for building a dynamic and responsive user interface).
- **Backend**: .NET 10 (C#), deployed within **Docker** containers to ensure consistent and isolated environments across development and production.
- **Database**: MongoDB (a NoSQL document database chosen for flexible and highly performant data persistence).
- **Key Frameworks & Libraries**:
  - *API Gateway*: `FastEndpoints` (for structuring RESTful routing with minimal overhead).
  - *Microservice Communication*: `Grpc.AspNetCore` and `Grpc.Tools` (for efficient, strongly-typed internal RPC calls).
  - *Data Access*: `MongoDB.EntityFrameworkCore` and the native driver.
  - *Security*: `System.IdentityModel.Tokens.Jwt` and `Microsoft.IdentityModel.Tokens` (for robust JWT-based authentication and authorization policies).

2 System Architecture

The system is designed using a modern distributed approach consisting of a central **API Gateway** and multiple specialized **Microservices**. This separation allows for independent scaling, technology flexibility, and clear boundaries between domains.

2.1 API Gateway Layer

The API Gateway acts as the entry point for all client applications, managing security, request routing, and protocol translation.

- **Routing & FastEndpoints**: Built using the FastEndpoints library, the gateway defines RESTful routes and translates incoming HTTP requests into gRPC calls for internal microservices.
- **Security & Policies**: Centralized authentication is handled via a custom Authentication Handler, which validates JWT tokens and enforces authorization policies such as Authenticated User or Moderator.
- **Response Aggregation**: A standardized base class manages how internal responses are returned to the client, mapping internal status codes to proper HTTP responses like 400 (Bad Request) or 404 (Not Found).

2.2 Microservices Architecture

Each microservice is an autonomous unit responsible for a specific domain (e.g., Users, Forum, Images) and follows a strictly layered internal structure.

2.2.1 Internal Microservice Layers

1. **Communication Level**: Handles gRPC interactions using a Message Dispatcher to route Protobuf requests to their specific logic handlers.
2. **Implementation & Business Logic**: Interfaces define the core capabilities, such as Authentication, Registration, and User Management, keeping the logic decoupled from the transport layer.
3. **Models & Persistence**: Contains database models and Data Transfer Objects (DTOs). Services utilize database abstractions for data persistence, often implementing optimized indexing for performance.

3 Persistence Layer (MongoDB)

The database layer is built around an abstraction of the MongoDB C# Driver to provide a clean, testable interface for business logic.

3.1 Core Data Abstractions

- **DatabaseModel**: An abstract record that enforces a standardized `ObjectId` representation for all persistent entities.
- **CollectionNameAttribute**: A custom attribute used to map C# classes to specific MongoDB collection names at runtime.
- **IDatabaseCollection**: A generic repository interface that provides standardized asynchronous operations such as `InsertAsync`, `UpsertAsync`, and predicate-based updates.

3.2 Infrastructure & Optimization

- **Database & Cache**: The `Database` class acts as a factory, caching collection wrappers in a `ConcurrentDictionary` to ensure thread-safe access and high performance.
- **AsyncLazy**: A utility class that uses `SemaphoreSlim` to ensure that heavy initialization tasks (like connecting to a collection and building indexes) are performed exactly once across multiple concurrent requests.
- **Dependency Injection**: An extension method registers the `IMongoClient`, `IMongoDatabase`, and the `IDatabase` singleton into the application container during startup.

4 Communication Layer (gRPC)

This project implements a Clean Architecture pattern specifically tailored for gRPC services in .NET. It utilizes a mediator-like pattern to decouple the gRPC service definitions from the actual business logic, ensuring that each request is processed through a standardized pipeline.

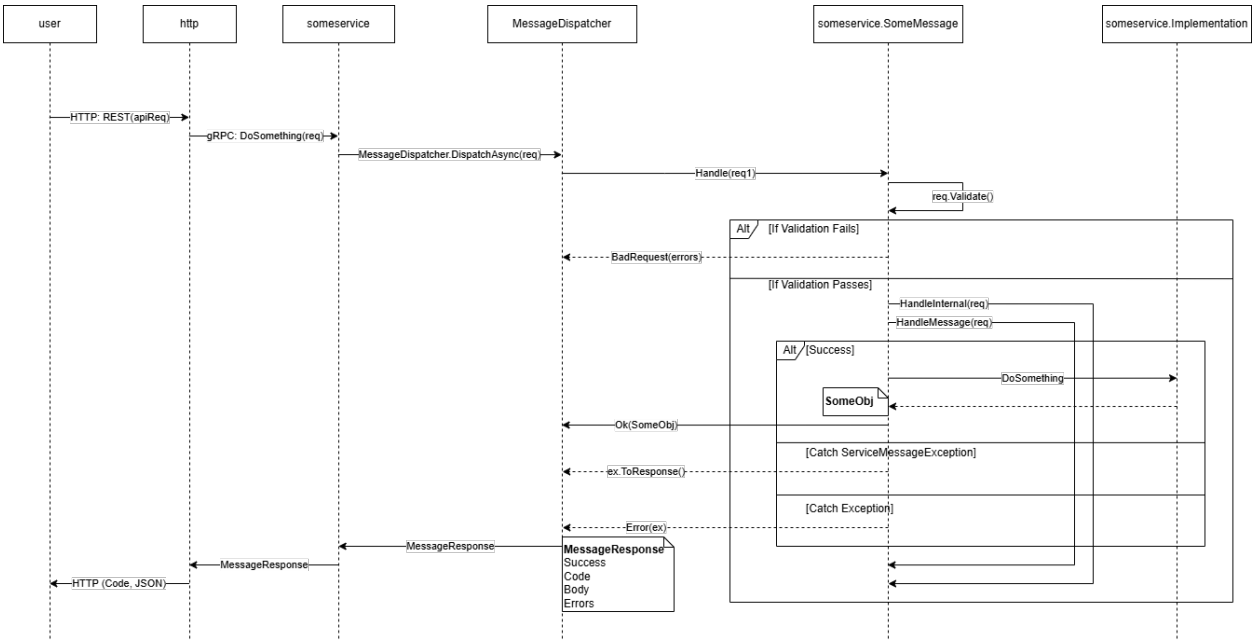


Figure 1: gRPC Request-Response Communication Flow

The sequence diagram above illustrates the request lifecycle. An incoming HTTP request from the client is translated by the Gateway into a gRPC message. This message is dispatched to the target microservice, where the `MessageDispatcher` dynamically resolves the appropriate handler. The base `Message` class intercepts the call to execute request validation, and then wraps the execution of the core business logic within a standardized `try-catch` block. Finally, the domain outcome (or exception) is packaged into a `MessageResponse` and propagated back to the API Gateway.

4.1 Messaging & Contract Layer

- **common.proto & MessageResponse.cs**: Define a unified internal communication contract across the entire ecosystem.
- **IRequestBase**: Ensures all internal requests are self-validating before they are processed by the business logic.

4.2 Handler Framework

- **MessageBase / Message**: Abstract classes that encapsulate cross-cutting concerns: validation, scoped logging, and standardized error handling.

- **ServiceMessageException**: A custom exception hierarchy that propagates errors from the business layer back to the gateway with specific semantic meaning.

4.3 Infrastructure

- **MessageDispatcher**: Decouples gRPC service definitions from their implementation, promoting highly readable and maintainable service classes.
- **MessageExtensions**: Provides automation for service registration using reflection to scan and register handlers into the dependency injection container.

5 Execution Flow: Authentication Example

The flow for authenticating a user demonstrates the integration of all layers in the system:

1. **Gateway**: Receives an HTTP POST request. The endpoint validates the payload and maps it to a gRPC request object.
2. **RPC Call**: The Gateway forwards the request to the Users Microservice via a generated gRPC client.
3. **Microservice Dispatch**: The Users Microservice receives the call and uses the Dispatcher to find the appropriate internal handler.
4. **Data Access**: The business logic handler uses an `AsyncLazy` collection reference to retrieve user data from MongoDB based on the provided email.
5. **Logic & Response**: The handler verifies the credentials, checks the user's ban status, generates a JWT, and returns a `MessageResponse`. The Gateway then maps this response into a standardized JSON object for the client.

6 Frontend Architecture & Application Flow

The frontend is built as a Single Page Application (SPA) using React, designed to provide a responsive, dynamic, and stateful user experience. It seamlessly integrates with the API Gateway to consume the backend microservices.

6.1 Core Technologies & Patterns

- **Framework:** React for component-based UI development.
- **Routing:** `react-router-dom` is used for client-side routing, enabling fluid navigation between pages without full browser reloads.
- **Styling:** Modular CSS (e.g., `Home.css`, `Auth.css`) ensures component-scoped styling and maintainability.
- **Service Layer:** API calls are abstracted into dedicated service modules (e.g., `questionService.js`, `userService.js`). This decouples the UI components from network logic and provides a clean interface for data fetching.

6.2 Authentication & State Management

The frontend relies on stateless JWT-based authentication. Upon successful login or registration, the backend returns a JWT and user metadata, which are stored in the browser's `localStorage` (`forum_token` and `forum_user`). Components parse this local storage to dynamically render context-aware UI elements, such as showing moderation tools to admins or restricting edit/delete actions to the original author of a post.

6.3 Core Pages & Components

6.3.1 Authentication (Login & Register)

Handles user onboarding and session creation. The forms feature controlled inputs and immediate error feedback. Upon success, they cache the credentials and redirect the user to the main dashboard.

6.3.2 Dashboard / Home

The central hub of the application. It fetches and displays a paginated list of questions.

- **Advanced Filtering:** Users can filter the feed by Title, Tag, Author Email, or view their own questions.
- **Hydration:** The component hydrates raw question data by fetching associated Author information and the current user's vote status to accurately reflect the upvote/downvote arrows.

6.3.3 Question Management (Ask & Detail)

- **Ask/Edit Question:** A reusable form component that handles both creating new questions and editing existing ones. It supports rich interactions, including image uploading with local blob previews and dynamic tag creation.
- **Question Detail:** A comprehensive view for a single question. It fetches the parent question, its image, and all nested comments. It encapsulates complex logic for casting votes, editing inline comments, and conditionally rendering edit/delete buttons based on the logged-in user's identity.

6.3.4 User & Role Management (Account & Moderator)

- **Account:** Allows users to manage their profile. They can view their total score, update their username, change their password, and permanently delete their account (Danger Zone).
- **Moderator:** A protected route designed for administrative control. It displays a tabular view of all registered users, allowing moderators to search for specific accounts and toggle ban/unban statuses with optional ban reasons.

6.4 Service Layer & API Integration

To maintain a clean separation of concerns, all network requests are abstracted into domain-specific service modules. This layer acts as the bridge between the React UI components and the backend API Gateway.

- **Core API Client (`apiClient.js`):** A centralized utility wrapper around the native `fetch` API. It automatically intercepts requests to attach the JWT token to the `Authorization` header, intelligently serializes payloads (handling both `application/json` and `FormData`), and standardizes error handling by parsing backend error responses into consistent UI-friendly exceptions.
- **Authentication Service (`authService.js`):** Manages user sessions by interfacing with the login and registration endpoints.
- **Question & Comment Services (`questionService.js`, `commentService.js`):** Encapsulate the core CRUD (Create, Read, Update, Delete) operations for the forum. The question service also handles paginated queries, tag resolution, and filtering algorithms (by title, tag, or author).
- **User Service (`userService.js`):** Handles profile management (updating username/password, retrieving scores, deleting accounts) as well as elevated moderator actions like fetching the entire user directory, banning, and unbanning users.
- **Vote Service (`voteService.js`):** Manages the upvote/downvote ecosystem, retrieving vote counts and determining the current user's interaction state with specific questions or comments.

- **Image Service (`imageService.js`):** Handles the complexities of file transmission. It uploads images using `FormData` and fetches protected images by attaching the authorization token and converting the binary response into a local browser Blob URL for rendering.

7 Testing Strategy

To ensure system reliability, maintainability, and functional correctness across the distributed microservice architecture, the project implements a multi-layered testing strategy.

7.1 Unit & Integration Testing (Frontend)

The frontend utilizes `Vitest` in conjunction with `React Testing Library` to validate component behavior in isolation.

- **Mocking:** Service layer dependencies (e.g., `authService`, `questionService`) are mocked using `vi.mock` to simulate API responses without requiring a running backend.
- **Component Verification:** Tests focus on user-centric interactions, such as form submissions, conditional rendering based on authentication state, and error handling for failed API requests.

7.2 End-to-End (E2E) Testing

E2E tests are implemented using `Cypress` to verify the integration between the Frontend, API Gateway, Microservices, and the MongoDB persistence layer.

- **Test Isolation:** Each test suite is designed to be autonomous. We utilize `cy.intercept` to manage network synchronization, ensuring the test runner waits for asynchronous API responses before proceeding to assertions.
- **Happy Path Coverage:** Tests focus on critical user flows—Registration, Authentication, and Question Creation—to ensure the "happy path" remains unobstructed during continuous development.
- **Network Reliability:** To mitigate issues related to gRPC request cancellation, tests implement request queueing strategies and deliberate synchronization points, ensuring that concurrent network requests do not exhaust connection pools at the API Gateway level.

7.3 Automated Quality Assurance

- **CI/CD Integration:** Automated testing pipelines are configured via GitHub Actions to execute the test suite on every pull request to identify regression issues immediately.

- **Linting & Static Analysis:** Automated linting workflows are enforced to maintain consistent code quality, adhere to naming conventions, and reduce technical debt across both frontend and backend repositories.